Redes de Computadores

Camada de Rede

Agenda

Algoritmos de Roteamento

Roteamento na Arquitetura TCP/IP

1

Algoritmos de Roteamento

Definições, Tipos, Implementações: Link State, Distance Vector, Roteamento Hierárquico

Definições

- Na camada de Rede é onde se determina o caminho que os pacotes percorrem entre remetentes e destinatários
  - Pacotes diferentes podem percorrer rotas diferentes para o mesmo par remetente e destinatário
  - Serviço de circuitos virtuais - pacotes diferentes percorrem a mesma rota para o mesmo par remetente e destinatário

Definições

- Um roteador faz a função de indexar a tabela de repasse e determina a interface de enlace para a qual o pacote deve ser encaminhado

- Algoritmos de roteamento trocam e calculam informações que são utilizadas para configurar as tabelas de repasse

Definições

- Roteador default
  - Também denominado roteador de 1º salto
  - Roteador ao qual está ligado o host remetente ou destinatário
    - Roteador da fonte: roteador default para o host remetente
    - Roteador do destino: roteador default para o host destinatário

Definições

- Rotear pacotes do **host remetente** para o **host destinatário** se reduz ao problema de rotear pacotes do **roteador da fonte** para o **roteador de destino**

- Dado um conjunto de roteadores conectados por enlaces, deseja-se descobrir um caminho entre o roteador de fonte e roteador de destino segundo uma função de custo

Definições

- Grafo G = (N,E): conjunto de N nós e E arestas no qual cada aresta é um par de nós do conjunto N
  - no contexto do roteamento da camada de rede, os nós do grafo representam roteadores e as arestas representam os enlaces físicos

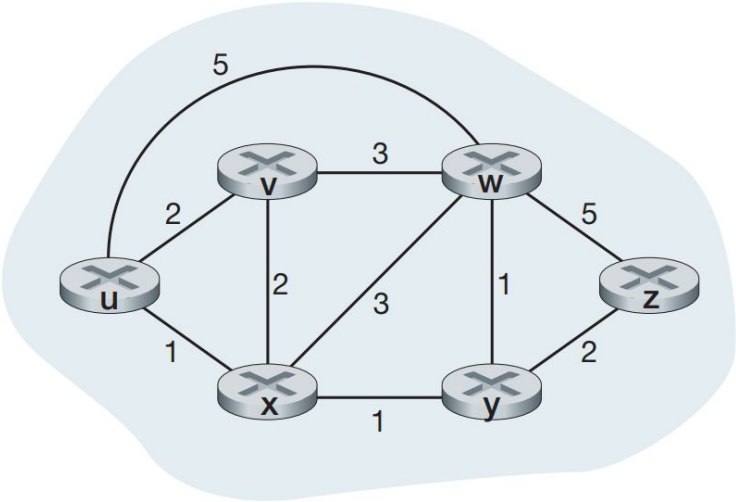
Exemplo

Seja o Grafo G = (N,E), onde:

N = { u, v, w, x, y, z } e

E = { (u,v), (u,x), (v,x), (v,w), (x,w),

(x,y), (w,y), (w,z), (y,z) }



Definições

- Caminho em um grafo: sequência de nós $(x_1, x_2, \ldots, x_p)$ tal que cada um dos pares

  $(x_1, x_2), (x_2, x_3), \ldots, (x_{p-1}, x_p)$ são arestas em E
- Para qualquer aresta (x,y) em E, c(x,y) representa o custo da aresta entre os nós x e y

Definições

- Para qualquer aresta (x,y) em E, c(x,y) representa o custo da aresta entre os nós x e y
  - Grafos são direcionais, ou seja, uma aresta (x,y) é igual a aresta (y,x), assim como c(x,y) é igual a c(y,x)
  - O custo do caminho $(x_1, x_2, \ldots, x_p)$ é a soma de todos os custos dos pares (x_k , x_{k+1}) ao longo do caminho, ou seja

$$c(x_1 , x_2) + \ldots + c(x_{p-1}, x_p)$$

Definições

- Se todas as arestas do grafo tiverem o mesmo custo, encontrar o caminho de menor custo significa encontrar o caminho no grafo com o menor número de enlaces entre fonte e destino
  - O custo pode ser inversamente relacionado à largura de banda do enlace ou proporcional ao congestionamento do mesmo
  - Diferentes algoritmos fornecem diferentes respostas em razão da função custo utilizada para encontrar o caminho de menor custo

Tipos de Algoritmo

- **Algoritmo de roteamento global**
  - Calcula-se o caminho de menor custo entre fonte e destino usando o conhecimento global da rede
  - Considera-se todos os nós e custos de todos os enlaces
  - São também denominados de **algoritmos de estado de enlace** ou **algoritmos com informação global de estado de enlace**

Tipos de Algoritmo

- **Algoritmo de roteamento descentralizado**
  - Calcula-se o caminho de menor custo usando o conhecimento parcial da rede, ou seja, não há informação dos custos de todos os enlaces
  - São também denominados de **algoritmo de vetor de distâncias**
    - Cada nó mantém um vetor de estimativas de custos (distâncias) de um nó até todos os outros nós da rede

Tipos de Algoritmo

- **Algoritmos de roteamento estático**: as rotas mudam muito lentamente ao longo do tempo, muitas vezes por intervenção humana
- **Algoritmos de roteamento dinâmico**: as rotas mudam à medida que mudam as cargas de tráfego ou a topologia da rede

Tipos de Algoritmo

- **Algoritmo sensível à carga**: custos dos enlaces variam dinamicamente para refletir o nível corrente de congestionamento
- **Algoritmo insensível à carga**: custos dos enlace não refletem explicitamente seu nível de congestionamento, assim, a escolha de rota não considera o congestionamento dos enlaces

Algoritmo Link-State

- Algoritmo de Dijkstra
- Tanto a topologia da rede quanto os custos dos enlaces são conhecidos por todos os nós
  - Normalmente, estas informações são encaminhadas a cada um dos nós através de broadcast de estado do enlace
  - Adicionalmente, todos os nós compartilha, o mesmo conjunto de informações da rede, permitindo que tenham uma visão idêntica

Algoritmo Link-State

- Algoritmo de Dijkstra
- Tanto a topologia da rede quanto os custos dos enlaces são conhecidos por todos os nós
    - Cada nó pode com base nas informações, rodar o algoritmo e assim calcular o conjunto de caminhos de menor custo para todos os outros nós

Algoritmo Link-State

- Algoritmo de Dijkstra
- Tanto a topologia da rede quanto os custos dos enlaces são conhecidos por todos os nós
    - O algoritmo iterativo que tem a propriedade de, após a k-ésima iteração, conhecer os caminhos de menor custo para k nós de destino, dentre os caminhos de menor custo até todos os nós de destino

Algoritmo Link-State

- Notação do Algoritmo de Dijkstra:
- **c(x,y) – custo do enlace do nó x a y**, normalmente é representado por um inteiro quando x e y são vizinhos e infinito se não vizinhos diretos
- **D(v) – valor atual** do caminho **de menor custo** entre o nó fonte e o nó destino v até a iteração corrente do algoritmo
- **p(v) – nó anterior ou vizinho de v** ao longo do caminho corrente de menor custo desde o nó fonte até o nó v
- **N' – subconjunto de nós de N** cujo **caminho** de **menor custo** é **conhecido**

Algoritmo Link-State

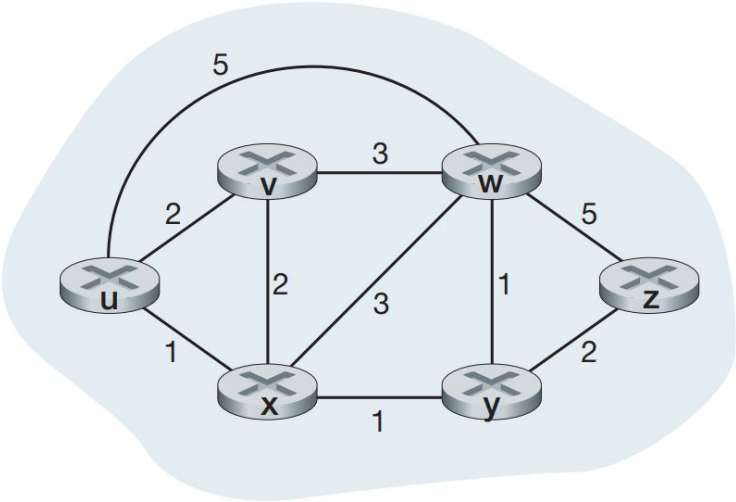
```
Inicialização
    N' = {u}
    para todos os nós v
        se v for um vizinho de u
            então D(v) = c(u,v)
        senão D(v) = ∞

Loop
    encontre w não em N' tal que D(w) é um mínimo
    adicione w a N'
    atualize D(v) para cada vizinho v de w e não em N':
            D(v) = min( D(v), D(w) + c(w,v) )
    /* o novo custo para v é o velho custo para v ou
        o custo do menor caminho conhecido para w mais o custo de w para v */
até N'= N
```

Exemplo

Como fica a tabela de repasse

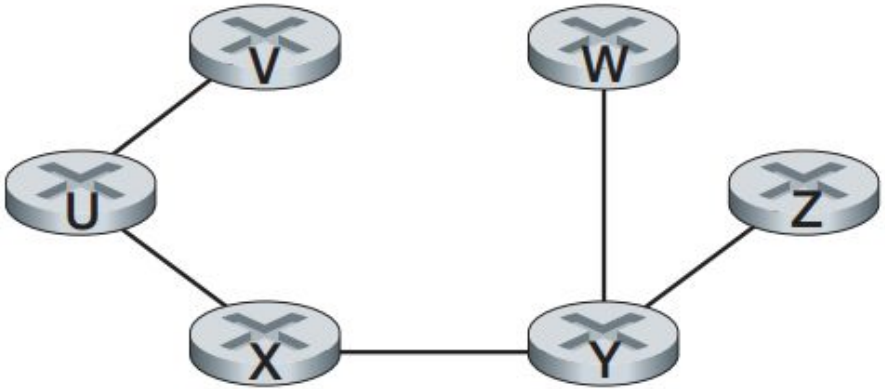
a partir do nó fonte **u**?



Exemplo

Etapa	*N'*	*D(v),p(v)*	*D(w),p(w)*	*D(x),p(x)*	*D(y),p(y)*	*D(z),p(z)*
0	u	2,u	5,u	1,u	∞	∞
1	ux	2,u	4,x		2,x	∞
2	uxy	2,u	3,y			4,y
3	uxyv		3,y			4,y
4	uxyvw					4,y
5	uxyvwz					

Exemplo



Destino	Enlace
v	(u, v)
w	(u, x)
x	(u, x)
y	(u, x)
z	(u, x)

Algoritmo Link-State

- A complexidade do algoritmo de Dijkstra é, em termos gerais, o número total de nós que precisamos pesquisar em todas as iterações

$$n * (n+1)/2$$

  - Para o pior caso é $O(n^2)$

Algoritmo Vetor de Distâncias

- Algoritmo distribuído
- Cada nó recebe informação parcial da topologia da rede
  - Nós vizinhos, sobre os quais calcula as rotas
  - Não requer que todos os nós rodem simultaneamente
  - A troca de informações acerca da topologia ocorre continuamente até que os nós vizinhos não tenham mais informações para trocar
    - Mudança de custos de enlace
    - Mensagens de atualização do Vetor do nó vizinho

Algoritmo Vetor de Distâncias

- Notação do Alg. de Vetor de Distâncias
- $d_x(y)$ – custo do caminho de menor custo do nó x ao nó y cujo valor é dado pela Equação de Bellman-Ford

  $\min_v \{ c(x,v) + d_v(y) \}$, onde $\min_v$ é calculado para todos os vizinhos de x

  - Ao transitarmos de x para y, o caminho de menor custo de x a y será o menor valor da expressão $c(x,v) + d_v(y)$ para todos os vizinhos v

Algoritmo Vetor de Distâncias

Para cada nó, *x*:

```
Inicialização:
    para todos os destinos y em N:
        Dx(y) = c(x,y) /* se y não é um vizinho então c(x,y) = ∞ */
para cada vizinho w
    Dw(y) = ? para todos os destinos y em N
para cada vizinho w
    envia vetor de distâncias Dx  = [Dx(y): y em N] para w

loop
    espere (até que ocorra uma mudança no custo do enlace ao vizinho
            w ou até a recepção de um vetor de distâncias do vizinho w)

    para cada y em N:
        Dx(y) = min_v{c(x,v) + Dv(y)}

    se Dx(y) mudou para algum destino y
        envia vetor de distâncias Dx = [Dx(y): y em N] para todos os vizinhos

para sempre
```

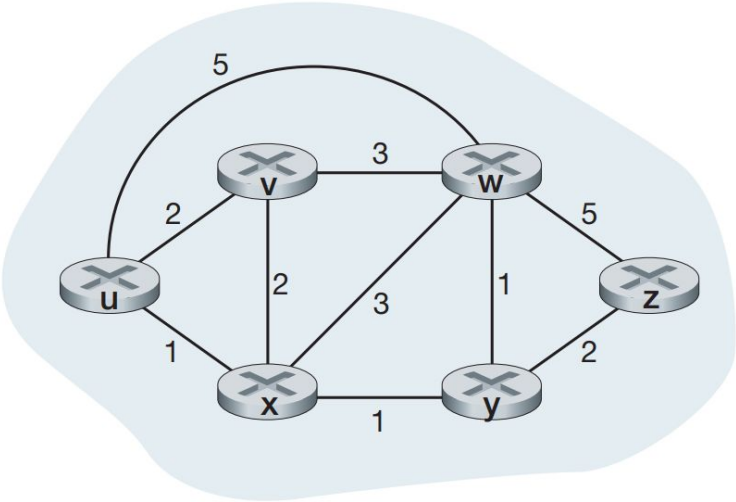
Exemplo

Seja o Grafo G = (N,E), onde:

N = { u, v, w, x, y, z } e

E = { (u,v), (u,x), (v,x), (v,w), (x,w),

(x,y), (w,y), (w,z), (y,z) }



Exemplo

- O nó u tem como vizinhos os nós v, x e w e para vários caminhos do grafo é fácil perceber que $d_v(z) = 5$; $d_x(z) = 3$; $d_w(z) = 3$
- Ao considerar estes valores na Eq. Bellman-Ford juntamente com $c(u,v) = 2$, $c(u,x) = 1$, e $c(u,w) = 5$ teremos o caminho de menor custo

$$d_u(z) = \min \{ 2+5; 1+3; 5+3 \} = 4$$

Algoritmo Vetor de Distâncias

- Equação de Bellman-Ford
  - Cada nó começa com $D_x(y)$, ou seja, uma estimativa do custo do caminho de menor custo entre ele mesmo e o nó y para todos os nós em N
  - $D_x = [\ D_x(y) : y \text{ em } N\ ]$ - vetor de distâncias do nó x, ou seja, vetor de estimativas de custo de x até todos os outros nós, y em N

Algoritmo Vetor de Distâncias

- Equação de Bellman-Ford
  - Cada nó mantém o custo para cada um dos seus vizinhos, ou seja, c(x,v) com o qual está diretamente ligado;
  - Cada nó mantém D_x = [D_x(y) : y em N] contendo a estimativa de x para seus custos até todos os destinos y em N
  - Cada nó mantém os vetores de distâncias de seus vizinhos, isto é, D_v = [D_v(y) : y em N] para cada vizinho v em N

Resumo até aqui...

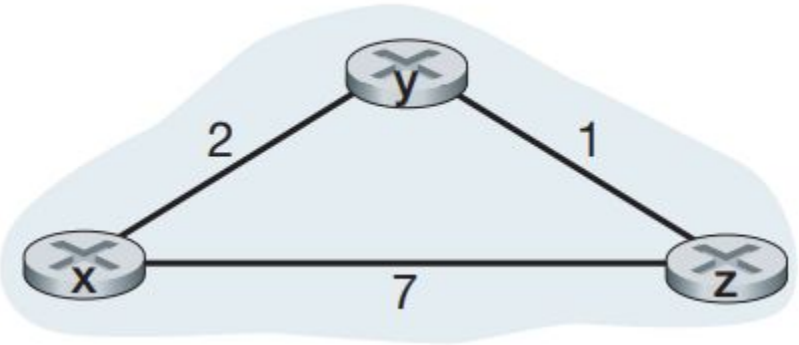
- **Link State** – algoritmo **global** no sentido de que requer que cada nó obtenha em primeiro lugar, um mapa completo da rede antes de rodar o Alg. de Dijkstra (Ex.: OSPF)
- **Vetor de Distâncias** – algoritmo **descentralizado** onde cada nó utiliza informações que recebe de seus vizinhos, ou seja, não há um mapa completo da rede antes de sua execução
    - Ex.: RIP; BGP; ISO IDRP, IPX da Novell; ARPAnet da Arq. TCP/IP

Resumo até aqui...

- **Link State**
  - Envia updates para toda a rede (broadcast)
- **Vetor de Distâncias**
  - Envia updates periódicos entre 30s a 90s
  - Somentes os vizinhos de um nó conhecem os custo do enlace
    - Por confiar nos vizinhos, podem vir a ter informações erradas

Exemplo

- Considere 03 nós interconectados por 03 enlaces cujos custos são apresentados na figura abaixo



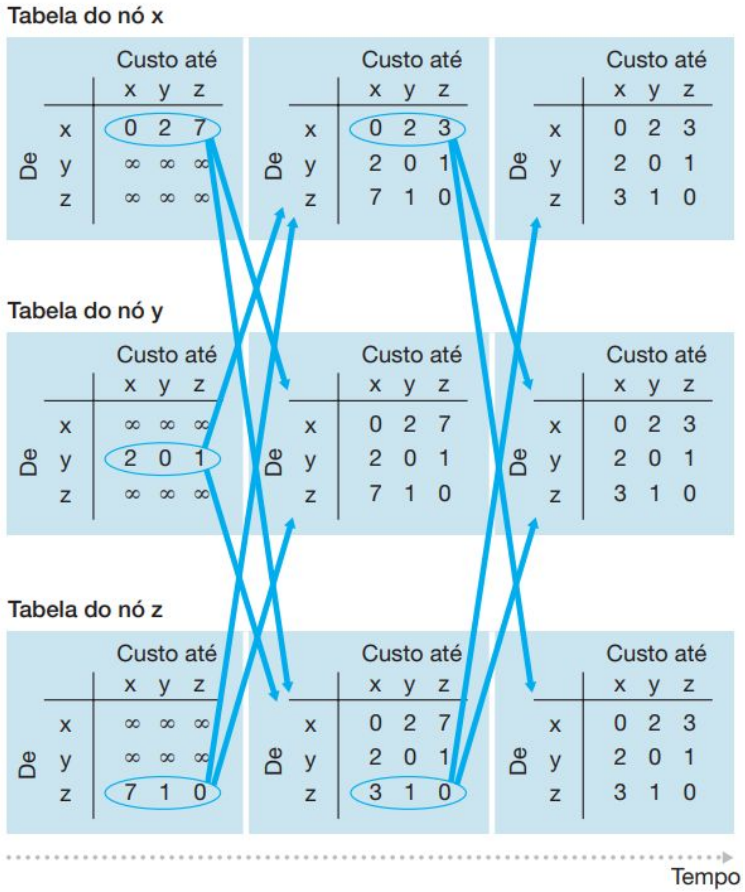


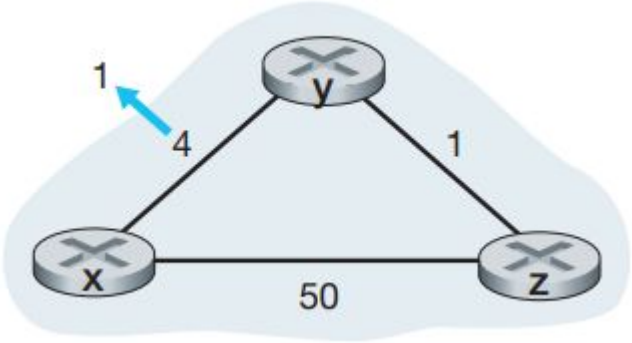
Tabela do nó x
Custo até
De
x y z
x 0 2 7
y ∞ ∞ ∞
z ∞ ∞ ∞
x 0 2 3
y 2 0 1
z 7 1 0
x 0 2 3
y 2 0 1
z 3 1 0
Tabela do nó y
x ∞ ∞ ∞
y 2 0 1
z ∞ ∞ ∞
x 0 2 7
y 2 0 1
z 7 1 0
x 0 2 3
y 2 0 1
z 3 1 0
Tabela do nó z
x ∞ ∞ ∞
y ∞ ∞ ∞
z 7 1 0
x 0 2 7
y 2 0 1
z 3 1 0
x 0 2 3
y 2 0 1
z 3 1 0
Tempo

Algoritmo Vetor de Distâncias

- Mudanças no Custo do Enlace e Falha no Enlace:
  - Quando um nó detecta uma mudança no custo de enlace até um vizinho (linhas 10-11), ele atualiza seu vetor de distâncias (linhas 13-14)
  - Se houver modificação no custo do caminho de menor custo, também informa a seus vizinhos (linhas 16-17) seu novo vetor de distâncias

Exemplo

- Considere 3 nós x, y e z interconectados por 3 enlaces cujos custos são:
  - c(x,y) = 4 ⟶ 1
  - c(y,z)= 1 e
  - c(x,z)= 50

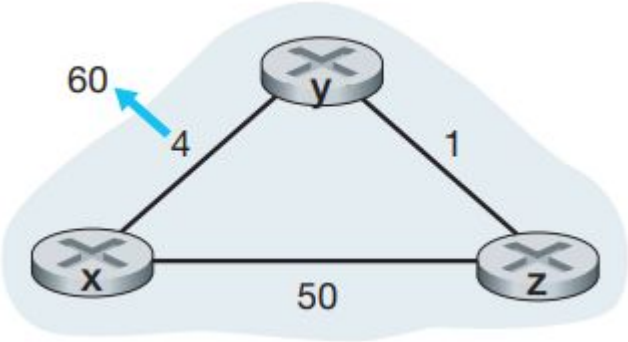


Exemplo

- O nó x detecta mudança no custo em t_o e, assim, atualiza seu vetor de distâncias e informa essa mudança a seus vizinhos
- No tempo t_1 z recebe a atualização de y, recalcula um novo custo para x, ou seja, 5 para 2 e envia um novo vetor aos vizinhos
- no tempo t_2 y recebe a atualização de z e, ao recalcular os custos, verifica que não mudaram, por conseguinte, não envia mensagem aos vizinhos

Exemplo

- Considere 3 nós x, y e z interconectados por 3 enlaces cujos custos são:
  - c(x,y) = 4 → 60
  - c(y,z)= 1 e
  - c(x,z)= 50



Exemplo

- O nó x detecta mudança no custo em t_o e, assim, atualiza seu vetor de distâncias e informa essa mudança a seus vizinhos
- O nó y sabe que o custo direto até x é 60 e que z disse a y que z pode chegar a x com um custo de 5 (custo via z está errado!)
- Em t_1 teremos um loop de roteamento, pois para chegar a x, y faz a rota através de z, que por sua vez faz a rota através de y

Exemplo

- O nó z recebe em t_2 o vetor de distâncias de y que indica que o custo mínimo de y até x é 6, mas z sabe que pode chegar a y com um custo de 1 e, assim, calcula um novo menor custo
- Como $D_z(x) = \min \{ 50 + 0 ; 1 + 6 \} = 7$, ou seja, custo de z a x aumentou, z informa ao nó y o seu novo vetor

Algoritmo Vetor de Distâncias

- Contagem ao infinito
  - O aumento do custo de enlace propagou-se muito devagar fazendo-se com que a convergência seja lenta
- O cenário descrito anteriormente pode ser evitado usando-se a técnica **poisoned reverse** (reversão envenenada)
  - Se o nó z passa por y para chegar a x, z informa a y que sua distância a x é infinita, assim, y não mais irá rotear para x por z
  - Limitação da técnica a dois nós imediatamente vizinhos

Comparação entre Algoritmos

- Complexidade da mensagem
  - Link-State - com N nós e E enlaces
    - Complexidade de O(N * E) para mensagens enviadas
  - Vetor de Distâncias - troca mensagens apenas entre vizinhos
    - O tempo convergência varia

Comparação entre Algoritmos

- Velocidade de convergência
  - Link-State
    - Algoritmo $O(N^2)$ requer $O(N*E)$ mensagens (pode ter oscilações)
  - Vetor de Distâncias
    - O tempo de convergência varia
    - Podem ter loops de roteamento bem como o problema da contagem até o infinito

Comparação entre Algoritmos

- Robustez (o que acontece se o roteador der defeito?)
  - Link-State
    - O nó pode anunciar custo incorreto do enlace
    - Cada nó calcula apenas sua própria tabela de repasse
  - Vetor de Distâncias
    - O nó pode anunciar custo incorreto do enlace e como a tabela de cada nó é utilizada por outros nós
    - O erro se propaga pela rede

Algoritmo Roteamento Hierárquico

- Considerou-se que o roteamento é ideal, ou seja, todos os roteadores são idênticos bem como o algoritmo de roteamento
  - Escalabilidade: Quanto maior o número de roteadores, maior será a sobrecarga de cálculo, armazenamento bem como o de comunicação de informações para a tabela de roteamento
    - Por exemplo, com 200 milhões de destinos, não se pode armazenar todos os destinos nas tabelas de roteamento
      - Uma troca de tabela de roteamento atolaria os enlaces

Algoritmo Roteamento Hierárquico

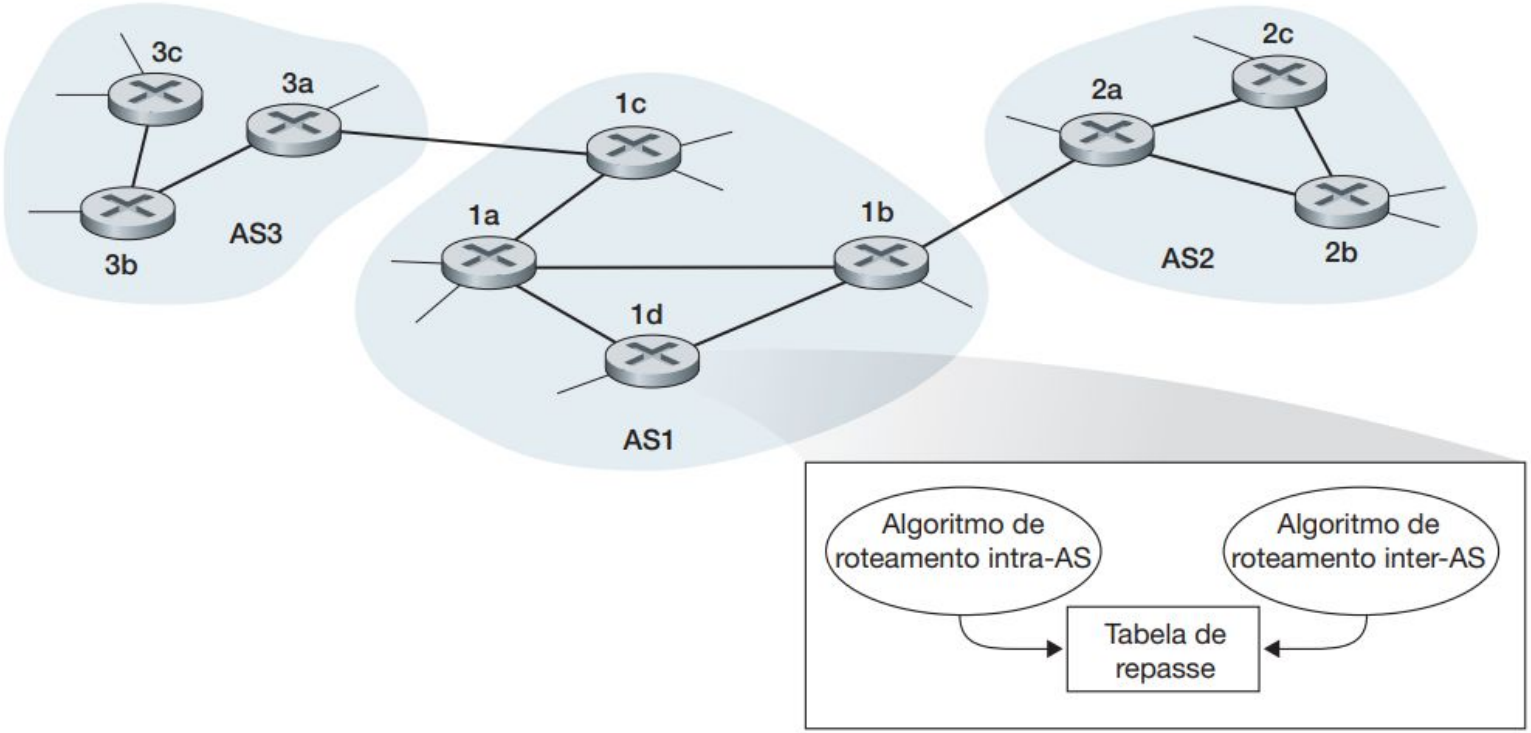
- Considerou-se que o roteamento é ideal, ou seja, todos os roteadores são idênticos bem como o algoritmo de roteamento
  - Autonomia administrativa
    - Autonomia na gestão de roteadores pelas empresas sob as quais são mantidos
    - Transparência ao público quanto a organização interna das redes são fatores que devem ser considerados

Algoritmo Roteamento Hierárquico

- Esses problemas podem ser resolvidos agrupando roteadores em sistemas autônomos (ASs),
  - Criação de grupos de roteadores sob o mesmo domínio administrativo
    - Por exemplo, mesmo ISP ou mesma Rede Corporativa

Algoritmo Roteamento Hierárquico

- Esses problemas podem ser resolvidos agrupando roteadores em sistemas autônomos (ASs),
  - Um algoritmo de roteamento dentro de um sistema autônomo é denominado Protocolo de Roteamento Intra-AS
  - Para conectar ASs entre si, um ou mais Roteadores de Borda em um AS tem a tarefa adicional de transmitir pacotes a destinos externos ao AS


3c
3a
3b
AS3
1c
1a
1b
1d
AS1
2c
2a
2b
AS2
Algoritmo de roteamento intra-AS
Algoritmo de roteamento inter-AS
Tabela de repasse

Algoritmo Roteamento Hierárquico

- Como um roteador que está dentro de algum AS saberá como rotear um pacote até um destino que está fora do AS?
  - Para os casos em que se tem **apenas um roteador de borda**, o algoritmo de roteamento Intra-AS já calculou o caminho de menor custo entre os roteadores internos e o roteador de borda do AS
  - Uma vez que o pacote seja roteado para fora do AS, através do roteador de borda, então o AS que está na outra extremidade do enlace assume o roteamento do pacote até o destino final

Exemplo

- Considere o roteamento de um pacote pelo roteador 2b para um destino fora do Sistema Autônomo AS2
  - O roteador 2b transmite o pacote para 2a, que por sua vez retransmitirá a 1b, portanto, trabalho de AS2 referente a este pacote está finalizado

Algoritmo Roteamento Hierárquico

- Como um roteador que está dentro de algum AS saberá como rotear um pacote até um destino que está fora do AS?
  - Para os casos em que um AS possui **2 ou mais enlaces que levam para fora do AS**, o repasse é significativamente mais desafiador
  - Embora a tarefa seja decidir para qual enlace de saída o pacote será repassado, tal decisão exige que cada AS saiba quais destinos são alcançáveis a partir de cada outro AS vizinho ou não

Algoritmo Roteamento Hierárquico

- Como um roteador que está dentro de algum AS saberá como rotear um pacote até um destino que está fora do AS?
  - Para os casos em que um AS possui **2 ou mais enlaces que levam para fora do AS**, o repasse é significativamente mais desafiador
  - As informações sobre condições de alcance de AS vizinhos bem como a propagação dessas informações para todos os roteadores internos são gerenciadas pelo Protocolo de Roteamento Inter-ASs

Algoritmo Roteamento Hierárquico

- Observação: Os ASs da Internet rodam o mesmo Protocolo de Roteamento Inter-ASs, ou seja, o BGP4 permitindo que os roteadores recebam informações de protocolos de roteamento Intra-ASs e Inter-ASs

Algoritmo Roteamento Hierárquico

- Etapas da adição de um destino fora do AS à tabela de repasse de um roteador

Aprende por um protocolo inter-AS que a sub-rede x pode ser alcançada via vários roteadores de borda. → Usa informações de roteamento do protocolo intra-AS para determinar custos de caminhos de menor custo para cada um dos roteadores de borda. → Roteamento da batata quente: escolhe o roteador de borda que tenha o menor custo. → Determina, pela tabela de repasse, a interface I que leva ao roteador de borda de menor custo. Adiciona (x,I) à tabela de repasse.

2

Roteamento na Arquitetura TCP/IP

Intra-AS: RIP e OSPF, Entre ASs: BGP

Roteamento Intra-AS

- Também conhecido como Interior Gateway Protocols (IGP)
- Protocolos de roteamento intra-AS mais comuns:
  - RIP: Routing Information Protocol
  - OSPF: Open Shortest Path First
  - IGRP: Interior Gateway roteamento Protocol
    - Proprietário da Cisco

Algoritmo RIP

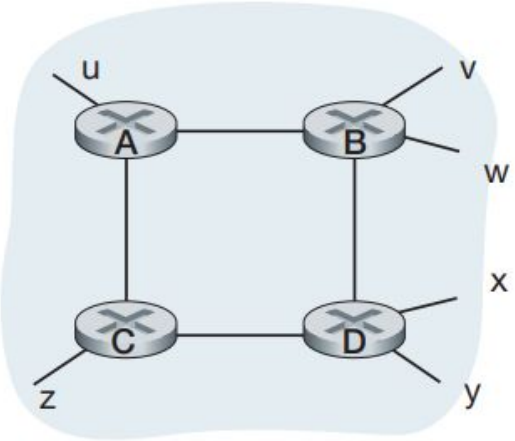
- Routing Information Protocol
- Algoritmo de vetor de distância
- Incluído na distribuição BSD-UNIX em 1982
- Métrica de distância: # de saltos (máx. = 15 saltos)

Algoritmo RIP

- Os vetores de distância são trocados entre vizinhos a cada 30s por meio de mensagem de resposta (anúncio)
  - Cada anúncio compreende uma lista de até 25 sub-redes de destino dentro do AS e as distâncias entre o remetente e cada uma dessas sub-redes
  - O roteador mantém uma tabela RIP (Tabela de Roteamento) que inclui o Vetor de Distâncias e a Tabela de Repasse

Exemplo

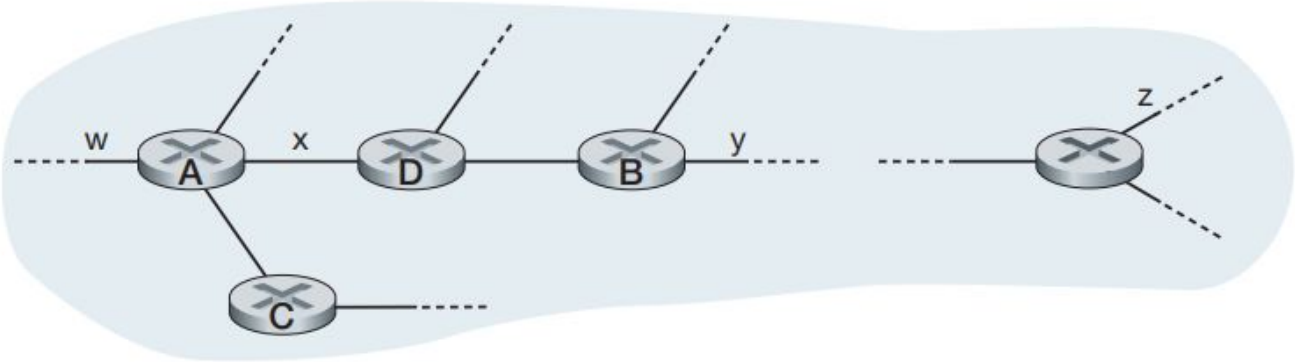
- Tabela de Repasse: Para o roteador A às sub-redes



Destino	Saltos
u	1
v	2
w	2
x	3
y	3
z	2

Exemplo

- Considere as sub-redes w, x, y e z



Exemplo

- Tabela de roteamento no roteador D antes de receber anúncio do roteador A

Sub-rede de destino	Roteador seguinte	Número de saltos até o destino
w	A	2
y	B	2
z	B	7
x	—	1
....	....	....

Exemplo

- Quando o roteador A anunciar sua tabela de rotas, o roteador D irá atualizar sua tabela de rotas para considerar os caminhos de menor custo, p.ex., sub-rede z ao custo 4

Exemplo

- Anúncio vindo do roteador A

Sub-rede de destino	Roteador seguinte	Número de saltos até o destino
z	C	4
w	—	1
x	—	1
....	....	....

Exemplo

- Tabela de roteamento no roteador D após receber anúncio do roteador A

Sub-rede de destino	Roteador seguinte	Número de saltos até o destino
w	A	2
y	B	2
z	A	5
....	....	....

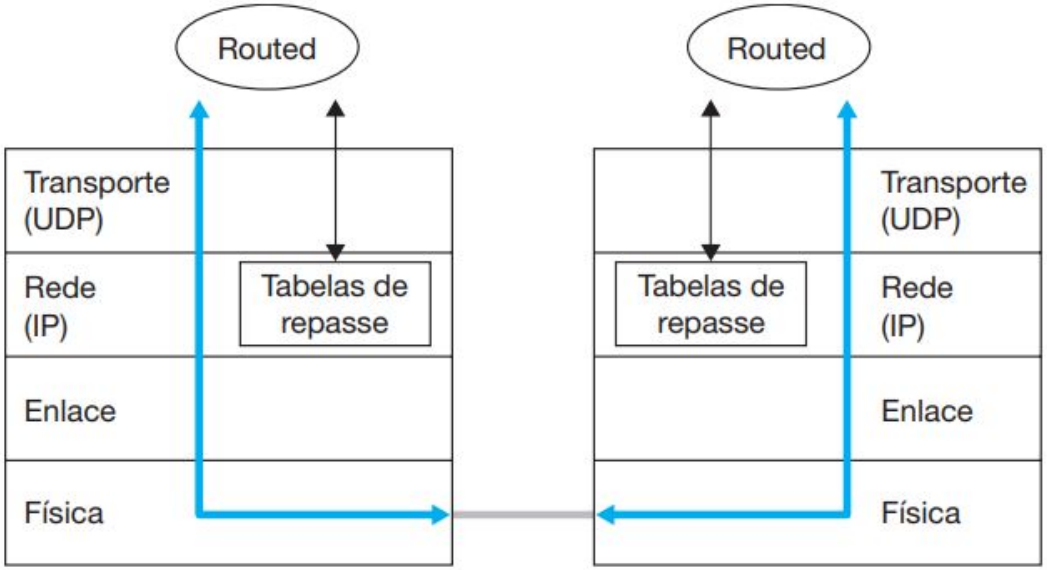
Algoritmo RIP

- Falha e Recuperação do enlace
  - Se nenhum anúncio for ouvido após 180s, então o vizinho/enlace é declarado morto
  - As rotas via vizinho são invalidadas
  - Novos anúncios enviados aos vizinhos
  - Os vizinhos por sua vez enviam novos anúncios (se não houver tabelas alteradas)

Algoritmo RIP

- Falha e Recuperação do enlace
  - A informação de falha do enlace rapidamente se propaga para rede inteira
  - Reversão envenenada é usada para impedir loops de pingue-pongue, considerando a distância infinita = 16 saltos
  - Os anúncios são enviados em pacotes UDP, repetidos periodicamente

Algoritmo RIP



Algoritmo OSPF

- Open Shortest Path First
- Está publicamente disponível em ISPs de níveis mais altos
  - RIP está em ISPs de níveis mais baixos
- Usa algoritmo Link State
  - Faz a disseminação de pacote LS
  - Mantém um mapa de topologia em cada nó
  - Calcula as rotas usando algoritmo de Dijkstra

Algoritmo OSPF

- Provê os mecanismos para determinar o caminho de roteamento de menor custo para um dado conjunto de pesos de enlaces
- Os anúncios são disseminados ao AS inteiro (com inundação-broadcasting)
    - São transportados nas mensagens OSPF diretamente por IP ao invés de TCP ou UDP

Algoritmo OSPF

- Recursos avançados
  - Segurança: todas as mensagens OSPF autenticadas visando impedir intrusão maliciosa
  - Há múltiplos caminhos de mesmo custo permitidos (no RIP, apenas um)
  - Atua como hierárquico em grandes domínios

Algoritmo OSPF

- Recursos avançados
  - Para cada enlace existem múltiplas métricas de custo para diferentes TOS
    - Por exemplo, custo de enlace de satélite definido baixo para melhor esforço; alto para tempo real
  - Oferece suporte integrado para uni e multicast
  - Multicast OSPF (MOSPF) usa a topologia do OSPF

Algoritmo OSPF

- Implementa transferência confiável de mensagem e transmissão broadcasting de estado de enlace
- Hierarquia em dois níveis: área local e backbone
  - Os anúncios de estado do enlace ocorrem somente na área
  - Cada nó tem topologia de área detalhada
    - Em outras áreas, somente direção conhecida (caminho mais curto)

Algoritmo OSPF

- Roteadores de borda: resumem distâncias às redes na própria área e anunciam para outros roteadores de borda
- Roteadores de backbone: executam roteamento OSPF limitado ao backbone – cujo papel principal é rotear tráfego entre as outras áreas do AS

Algoritmo OSPF

- Dessa forma, o roteamento entre áreas requer que:
  1. O pacote seja roteado até um roteador de borda da área
  2. Em seguida, o pacote deve seguir pelo roteador de backbone até o roteador de borda da área de destino
  3. Daí, segue até o destino final

Algoritmo OSPF

- Os roteadores são autenticados, garantido que apenas roteadores de confiança participem do Protocolo OSPF dentro de um AS
- Os caminhos múltiplos possuem custos iguais
  - Se há vários caminhos até o destino, não é preciso escolher um caminho único para carregar todo o tráfego
- Há suporte para unicast e multicast: OSPF Multicast fornece extensões simples ao OSPF para prover Roteamento Multicast

Algoritmo BGP

- Border Gateway Protocol
- Padrão de roteamento entre ASs na Internet
  - Permite que a sub-rede anuncie sua existência ao resto da Internet usando prefixos de sub-rede
- Especifica como são determinados os caminhos entre fonte e destino que abrangem vários sistemas autônomos

Algoritmo BGP

- Oferece a cada AS um meio de:
  - Obter informação de acessibilidade da sub-rede a partir de ASs vizinhos
  - Propagar informação de acessibilidade a todos os roteadores internos ao AS
  - Determinar rotas boas para sub-redes com base na informação e política de acessibilidade

Algoritmo BGP

- Pares de roteadores (pares BGP) trocam informações de roteamento nas conexões TCP semipermanentes (porta 179) - Sessões BGP
  - Sessão BGP é a conexão TCP juntamente com todas as mensagens BGP trocadas na conexão
  - Sessões BGP não precisam corresponder a enlaces físicos

Algoritmo BGP

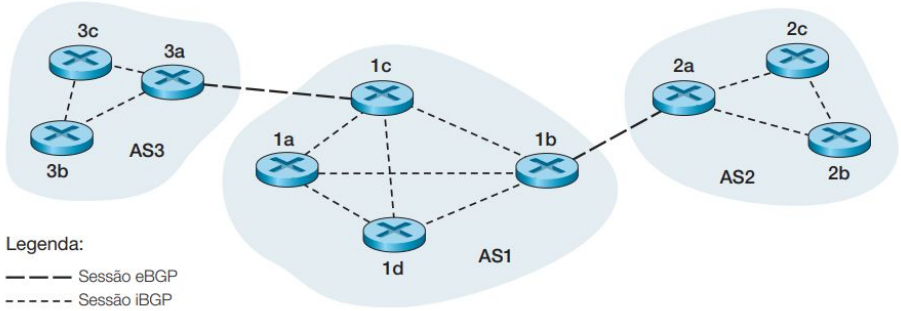
- Sessão BGP Externa (eBGP): sessão BGP que abrange 02 sistemas autônomos diferentes através dos roteadores de borda
- Sessão BGP Interna (iBGP): sessão BGP que abrange 02 sistemas autônomos diferentes através dos roteadores de borda

Algoritmo BGP

- eBGP e iBGP podem conhecer mais do que uma rota para qualquer prefixo determinado, e neste caso cabe ao BGP selecionar uma das rotas possíveis
  - Essas rotas recebem como um de seus atributos um valor de preferência local, permitindo-se que seja selecionado rotas com valores mais altos

Algoritmo BGP

- Exemplo: quando AS2 anuncia um prefixo para AS1
  - AS2 promete que repassará datagramas para esse prefixo
  - AS2 pode agregar prefixos em seu anúncio



Algoritmo BGP

- Usando sessão eBGP entre 3a e 1c, AS3 envia informação de atingibilidade do prefixo a AS1
  - 1c pode então usar iBGP para distribuir nova informação de prefixo a todos os roteadores em AS1
  - 1b pode então re-anunciar nova informação de atingibilidade para AS2 por sessão eBGP 1b-para-2a
  - Quando o roteador descobre um novo prefixo, ele cria entrada para prefixo em sua tabela de repasse

Algoritmo BGP

- Atributos e rotas de caminho
  - Os prefixos anunciados incluem atributos BGP
    - prefixo + atributos = rota
  - ASs são identificados por um número de AS globalmente exclusivo (ICANN de Registros)
  - Ao receber um anúncio de rota, o roteador de borda usa da sua política de importação para aceitar/declinar

Algoritmo BGP

- Atributos importantes:
  - AS-PATH: contém ASs através dos quais o anúncio do prefixo passou
    - Por exemplo, AS 67, AS 17
  - NEXT-HOP: indica um roteador específico do AS interno para AS do próximo salto
    - Podem ser múltiplos enlaces para AS atual até AS do próximo salto)

Algoritmo BGP

- Um roteador pode aprender sobre mais de 1 rota para algum prefixo, sendo necessário escolher uma rota

Algoritmo BGP

Regras de Eliminação:

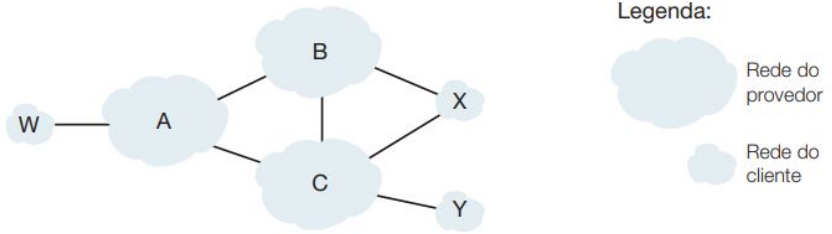
1. As rotas recebem, como um dos atributos, um valor de preferência local;
2. No caso de rotas remanescentes, seleciona-se a rota que tenha o AS-PATH mais curto;
3. Dentre as remanescentes, seleciona-se a que tenha o roteador NEXT-HOP mais próximo – menor custo de caminho de menor custo (roteamento batata-quente);
4. Uso de critérios adicionais: o roteador usa identificadores BGP para selecionar a rota.

Algoritmo BGP

- Mensagens BGP trocadas usando TCP
  - OPEN: abre conexão TCP com par e autentica remetente
  - UPDATE: anuncia novo caminho (ou retira antigo)
  - KEEPALIVE mantém conexão viva na ausência de UPDATES; também envia ACK para solicitação OPEN
  - NOTIFICATION: informa erros na msg anterior; também usada para fechar conexão

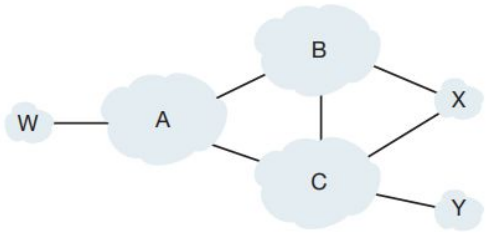
Exemplo

- Política de roteamento BGP:
- X é dual-homed: conectada a duas redes
- X não quer rotear a partir de B por meio de X para C
- Logo, X não anunciará a B uma rota para C (Política Seletiva de Anúncio de Rota)



Exemplo

- Política de roteamento BGP:
- A anuncia caminho AW para B
- B anuncia caminho BAW para X
- B deve anunciar caminho BAW para C? De forma alguma!
  - B não recebe retorno para roteamento CBAW
  - Nem W nem C são clientes de B
  - B quer forçar C a rotear para W por meio de A
  - B quer rotear apenas para/de seus clientes



Obrigado!

Dúvidas?